Vehicle Speed Prediction Using CNN-LSTM

Hui Li

1. Introduction

Estimating vehicle speed from monocular image sequences is a challenging task in autonomous driving, as it requires capturing subtle temporal changes between consecutive frames. Unlike traditional sensor-based approaches, vision-based speed estimation relies solely on visual motion cues, which are often noisy and sensitive to environmental variations.

This work proposes a hybrid deep learning model that combines a convolutional neural network (CNN) with a Long Short-Term Memory (LSTM) network (Hochreiter & Schmidhuber, 1997) to predict vehicle speed from image sequences. The CNN component extracts spatial features from each frame, while the LSTM captures temporal dependencies across consecutive frames.

The primary objective of this study is to evaluate how different hyperparameters and training strategies influence prediction accuracy and model stability, measured using Mean Absolute Error (MAE).

Additionally, this study also focuses on the practical workflow of building such a system, including data preprocessing, model debugging, and hyperparameter tuning.

2. Data Handling and Preprocessing

2.1 Dataset and Extraction

The dataset was extracted from ROS bag files using a custom Python pipeline. Images were extracted from the camera topic, and speed values were extracted from the speed topic. The extracted data was synchronized using nearest timestamp matching.
The processed dataset was saved as a CSV file containing image paths and corresponding speed values.

2.2 Data Preprocessing

Each sample consists of a sequence of consecutive frames, with the target speed defined as the speed of the final frame in the sequence.
Images were resized to 224×224 pixels and normalized using ImageNet statistics.
Additionally, data augmentation (ColorJitter) was applied during training to improve robustness to lighting variations.
Speed values were normalized using a maximum speed constant (SPEED_MAX = 3.0), mapping all targets into a range approximately between 0 and 1.

The dataset was split chronologically into training (70%), validation (15%), and testing (15%) sets to preserve temporal consistency.

2.3 Pipeline Verification with Small Dataset

Before training the final model, a smaller dataset from Moodle was used to verify the full pipeline, including data extraction, alignment, dataset splitting, and model training. This step ensured that all components of the system were functioning correctly before scaling to the full dataset.

3. Model Architecture

The proposed model combines a convolutional neural network (CNN) backbone with a Long Short-Term Memory (LSTM) network for temporal modeling. Specifically, a ResNet18-based architecture (He et al., 2016) is adopted as the feature extractor due to its strong representation capability in visual tasks.

The CNN component extracts a 512-dimensional feature vector from each input frame. These features are then fed into an LSTM network (Hochreiter & Schmidhuber, 1997) to capture temporal dependencies across the image sequence. The final hidden state of the LSTM is used as a compact representation of the sequence and is passed to a fully connected layer for speed regression.

To reduce overfitting, dropout is applied before the final regression layer. During training, the CNN backbone is frozen to leverage pretrained features and focus learning on the temporal modeling component.

4. Baseline Model and Debugging

A baseline CNN–LSTM model was first implemented to verify that the training pipeline worked correctly. During initial training, a dimension mismatch error occurred between the CNN output and LSTM input (input.size(-1) mismatch). This was resolved by adjusting the LSTM input size to match the CNN feature dimension. After fixing this issue, the model was able to train successfully.

5. Training Strategy

The model was trained using the SmoothL1 (Huber) loss function, which provides robustness to outliers compared to Mean Squared Error.

An Adam optimizer was used with a learning rate of 1e-5 and weight decay for regularization. A ReduceLROnPlateau scheduler dynamically reduced the learning rate when validation performance plateaued.

Gradient clipping was applied to prevent instability in LSTM training. Early stopping was used to avoid overfitting, based on validation MAE.

Multiple experiments were conducted to evaluate the effects of sequence length, loss functions, and CNN freezing strategies.

6. Results and Evaluation

6.1 Hyperparameter Experiments

Table 1: Overview of Hyperparameter Experiment Settings and Objectives

Experiment	SEQ_LEN	hidden_size	lr	Purpose
baseline	5	128	0.0001	Completed
Exp1	10	128	0.0001	Check whether longer time sequences perform better
Exp2	5	64	0.0001	Reduce model size to mitigate overfitting
Exp3	5	128	0.0005	Increase learning rate to see if convergence is faster

This table summarizes a set of comparative experiments designed for model optimization. Using the baseline configuration as a reference, each experiment explores a different key factor: sequence length (SEQ_LEN), model size (hidden_size), and learning rate (lr). Each experiment (Exp1–Exp3) modifies only one variable at a time to evaluate its impact on model performance, convergence speed, and overfitting risk, providing guidance for further hyperparameter tuning.

6.2 Final Model Repeated Runs

Table 2: Test Results Across Runs

Run	Val MAE	Test MAE
Run1	0.233855	0.356313
Run2	0.226567	0.356313
Run3	0.228076	0.350752

The best validation MAE values are consistently around 0.23, while the corresponding test MAE values are 0.356, 0.356, and 0.351, resulting in an average test MAE of approximately 0.354.

The limited variation among these results indicates that the proposed model exhibits stable and reproducible performance across repeated experiments.

This further suggests that the training process is robust and not significantly affected by random initialization.

It also indicates that the model generalizes well and produces reliable predictions across different runs.

6.3 Prediction Curve

Figure 1. Prediction vs Actual Speed

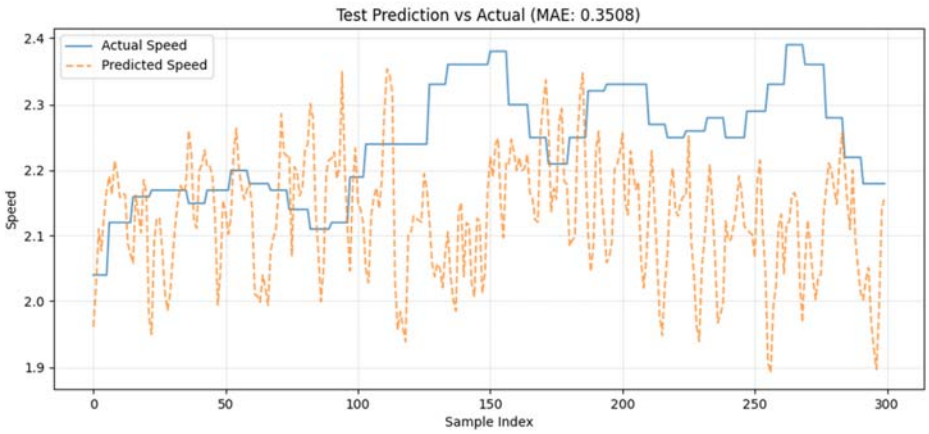


Figure 1 shows that the predicted speed closely follows the overall trend of the ground truth values, although minor fluctuations are present.

6.4 Scatter Plot

Figure 2. Prediction Scatter Plot

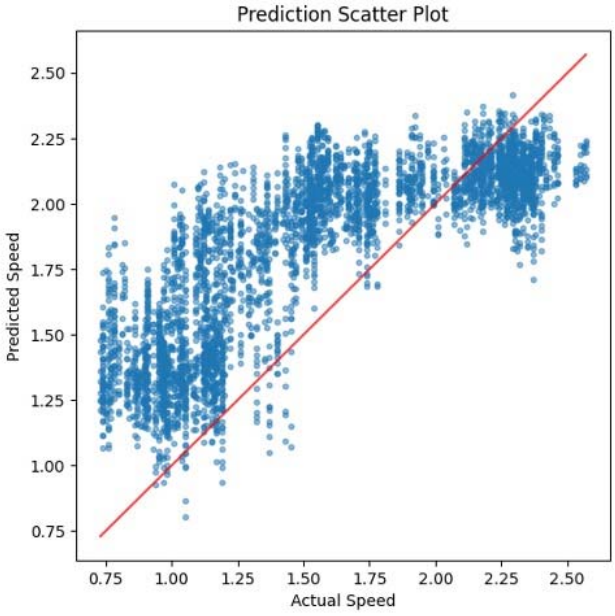


Figure 2 shows a strong correlation between predicted and actual speeds, with slight underestimation at higher speeds.

6.5 Loss Curve

Figure 3. Training and Validation Loss

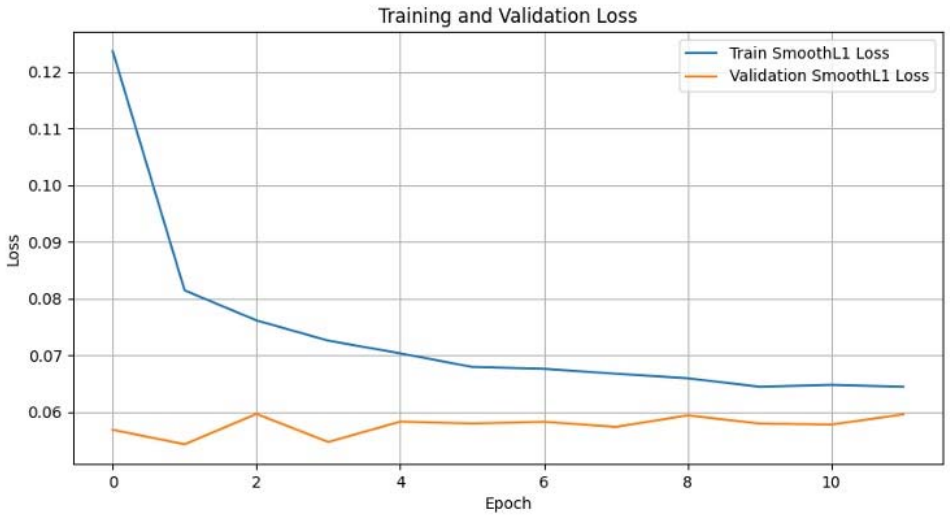


Figure 3 shows that the training and validation loss decrease smoothly, indicating stable convergence without severe overfitting.

6.6 MAE Curve

Figure 4. Validation MAE Curve

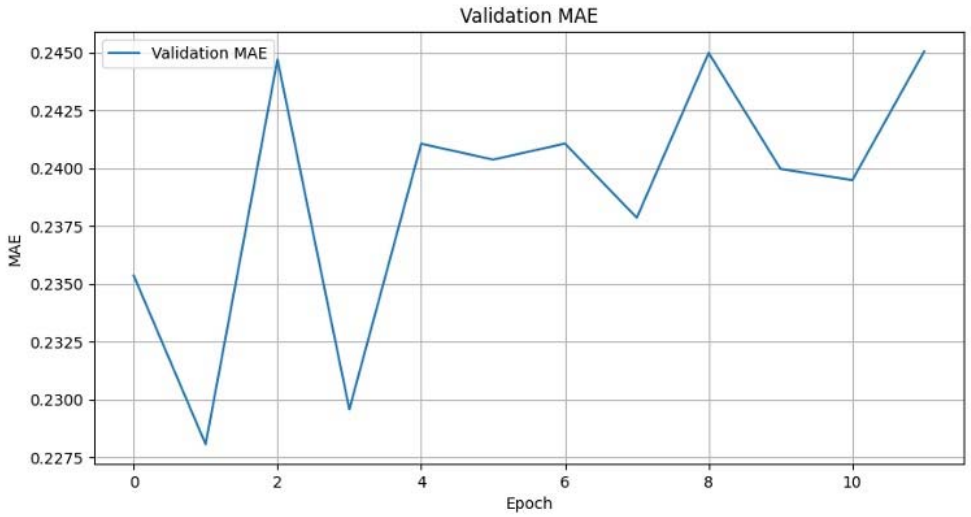


Figure 4 shows that the validation MAE stabilizes early, suggesting that the model quickly reaches its optimal performance.

7. Discussion

7.1 Experimental Findings

The experimental results provide several key insights into the proposed model.

Sequence length introduces a clear trade-off between prediction smoothness and responsiveness. Longer sequences improve temporal smoothness through implicit averaging across frames but reduce sensitivity to rapid speed changes, leading to higher MAE. In contrast, shorter sequences better capture local motion dynamics and achieve higher accuracy.

The choice of loss function also affects model robustness. SmoothL1 loss reduces the influence of outliers compared to MSE, resulting in more stable training.

Freezing the CNN backbone helps mitigate overfitting and reduces computational complexity. However, some systematic bias remains at higher speeds, indicating limitations in modeling fine-grained motion.

Overall, the model demonstrates a balance between accuracy and temporal stability, which is critical for real-world applications such as autonomous driving.

7.2 Challenges and Solutions

Several practical challenges were encountered during implementation, including Docker file system issues, Python indentation errors, and model dimension mismatches. These were resolved through systematic debugging and iterative testing.

Despite achieving stable training, the model remains sensitive to hyperparameter settings and initialization, highlighting the importance of careful experimental design.

8. Conclusion

This project demonstrates the effectiveness of combining CNN and LSTM architectures for vehicle speed prediction from monocular image sequences. The CNN–LSTM model successfully predicts vehicle speed from image sequences, achieving consistent performance across multiple runs.

While the model captures general motion trends effectively, further improvements could be achieved by incorporating optical flow or fine-tuning the CNN backbone.

The study further highlights the importance of temporal modeling, data normalization, hyperparameter tuning, and systematic debugging in sequence-based prediction tasks.

All experiments were conducted using PyTorch (Paszke et al., 2019) on a GPU-enabled (CUDA) environment. These findings provide a solid foundation for future research in vision-based vehicle speed estimation.

9. References

[1] He, K., Zhang, X., Ren, S., & Sun, J. (2016).

Deep Residual Learning for Image Recognition.

Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

[2] Hochreiter, S., & Schmidhuber, J. (1997).

Long Short-Term Memory.

Neural Computation, 9(8), 1735–1780.

[3] Udacity. (2016).

Self-Driving Car Steering Models Repository.

Available at: https://github.com/udacity/self-driving-car/tree/master/steering-models

[4] Paszke, A., et al. (2019).

PyTorch: An Imperative Style, High-Performance Deep Learning Library.

Advances in Neural Information Processing Systems (NeurIPS).

[5] torchvision.models — PyTorch Documentation.

Available at: https://pytorch.org/vision/stable/models.html

[6] Goodfellow, I., Bengio, Y., & Courville, A. (2016).

Deep Learning.

MIT Press.